DEFESA DE PROJETO UML

# Roteiro de Apresentação

## Pessoa 1 Introdução e Catálogo

Nosso objetivo hoje é apresentar como funciona o Sistema de Gerenciamento de Biblioteca, focando nas operações reais de quem utiliza e de quem administra o espaço. Vamos explicar a estrutura do sistema acompanhando a jornada de um livro e de um leitor.

Tudo começa no momento do acesso. Quando alguém tenta entrar no sistema, quem atua é o `ServicoAutenticacao`. Ele possui um método chamado `autenticar(login, senha)` que verifica as credenciais digitadas. Por questão de segurança, a senha é armazenada como um "hash", ou seja, de forma criptografada. Se os dados estiverem corretos, o serviço libera o acesso devolvendo os dados do usuário.

No sistema, existe uma classe geral chamada `Usuario`. Ela concentra as informações que todos precisam ter: nome, e-mail, login, senha e uma situação. A situação do usuário é controlada por uma lista fixa, chamada `SituacaoUsuario`, que define se a pessoa está no estado **ATIVO**, **BLOQUEADO** ou **COM_PENDENCIA**. A partir desse usuário geral, o sistema usa o que chamamos de herança para criar dois perfis específicos. O primeiro é o `Leitor`, que representa o aluno ou cliente, identificado por uma matrícula. O segundo é o `Bibliotecario`, o funcionário que administra a biblioteca, identificado por uma matrícula funcional. Eles herdam os dados básicos, mas possuem permissões diferentes.

Imaginando que um leitor acabou de fazer login, o próximo passo geralmente é procurar um livro. Ele usa o método `consultarCatalogo(criterio)`. Essa busca acontece no `Catalogo`, que é a classe responsável por organizar as obras. Aqui existe um conceito fundamental do nosso modelo: a diferença entre **Obra** e **Exemplar**.

A `Obra` representa a ideia intelectual do livro, como o título "Harry Potter", contendo o autor, ISBN e ano de publicação. Já o `Exemplar` é o livro físico, a cópia de papel que fica na prateleira. Cada exemplar possui um código tombo, que funciona como uma etiqueta ou código de barras único. Uma única obra pode possuir várias cópias físicas. Essa relação é chamada de agregação, um vínculo fraco onde a obra existe independentemente dos exemplares físicos. O leitor pode visualizar a obra por meio de `consultarDetalhes(idObra)` e descobrir quantas cópias existem pelo método `totalDisponiveis()`.

> *É importante notar que cada cópia física tem sua própria situação, definida pela enumeração SituacaoExemplar: ela pode estar DISPONIVEL, EMPRESTADO, DANIFICADO ou EXTRAVIADO. Portanto, achar o livro no catálogo não garante que a cópia esteja livre. Agora que o leitor encontrou a obra, como ele faz para levar a cópia para casa? Vou passar a palavra para **[Nome da Pessoa 2]**, que vai explicar o funcionamento dos empréstimos.*

## Pessoa 2 Empréstimos e Renovações

Dando continuidade, quando o leitor encontra a obra, ele vai até a prateleira pegar o exemplar físico. Quem administra esses itens é o `Bibliotecario`. Se a biblioteca comprar livros novos, é o bibliotecário que usa o método `cadastrarExemplar(obra, codigoTombo)` para inserir as novas cópias no sistema, ou `editarExemplar(exemplar)` para atualizar seus dados.

Para realizar a retirada do livro, entra em ação o "cérebro" das regras de negócio: o `ServicoEmprestimo`. Ele gerencia todas as operações de circulação. O bibliotecário inicia a operação acionando o método `emprestar(leitor, exemplar)` nesse serviço.

Antes de autorizar o empréstimo, o sistema faz duas verificações independentes. Primeiro, usa `verificarDisponibilidade(exemplar)` para garantir que a cópia física exata está com o status **DISPONIVEL**. Segundo, usa `verificarSituacaoUsuario(leitor)`. Dentro da classe Leitor, existe o método `estaApto()`, que confirma se o usuário está liberado para pegar livros, ou seja, se não está com status BLOQUEADO ou COM_PENDENCIA.

Se ambas as verificações passarem, o sistema cria um registro na classe `Emprestimo`. Esse registro conecta aquele leitor específico àquele exemplar físico específico. O empréstimo guarda a data de empréstimo, calcula a data de devolução prevista e marca o atributo ativo como verdadeiro. Simultaneamente, o sistema altera a situação do exemplar de disponível para **EMPRESTADO**.

Se o prazo estiver acabando e o leitor precisar de mais tempo, ele pode solicitar uma renovação. O sistema chama o método `renovar(emprestimo)` no serviço de empréstimo. Isso cria um novo registro na classe `Renovacao`. Essa relação é uma composição forte: a renovação só existe atrelada àquele empréstimo original. O registro de renovação salva a data da renovação (o dia do pedido) e estabelece uma nova data prevista de entrega. O material de estudo não define um limite máximo de renovações, indicando que um empréstimo pode ter várias renovações registradas.

> *Mas, eventualmente, o prazo acaba e o exemplar precisa retornar. Vou passar para **[Nome da Pessoa 3]**, que vai explicar como o sistema lida com devoluções, atrasos e os possíveis problemas desse processo.*

## Pessoa 3 Devoluções e Multas

Exatamente. O ciclo do empréstimo termina na devolução. Quando o leitor traz o livro de volta, o bibliotecário utiliza o serviço de empréstimo e aciona o método `devolver(emprestimo)`.

Essa ação cria um registro na classe `Devolucao`. A devolução encerra o empréstimo, mudando seu atributo ativo para falso. Neste momento, o sistema executa o método `calcularAtraso()`. Ele compara a data efetiva da devolução com a data prevista e gera a quantidade de dias de atraso. O método `estaAtrasado()` na classe Emprestimo ajuda a identificar se a data atual já ultrapassou o limite.

Se houver atraso, o `ServicoMulta` entra em ação, chamando `aplicarMulta(devolucao)`. Isso gera um registro na classe `Multa`, que guarda o valor, o motivo (como a quantidade de dias), a data de geração e um marcador booleano indicando se já foi paga. Cada devolução gera no máximo uma multa, mas o leitor possui uma classe chamada `ContaDebitos` associada ao seu cadastro. Essa conta acumula as multas recebidas pelo método `adicionarMulta(multa)` e atualiza o saldo devedor.

É essa conta de débitos que interage com a situação do leitor. Se ele tiver saldo devedor apontado no método `validarDebitos()`, sua situação muda para **COM_PENDENCIA**, o que o impede de pegar novos livros quando o método `estaApto()` for testado. Quando o leitor quita a dívida, o sistema usa `debitarMulta(multa)` no serviço de multas para dar baixa. Vale notar que o diagrama não estipula valores fixos de tarifas, pois isso depende das regras internas da biblioteca.

Além de multas por atraso, o livro pode retornar danificado. Se isso acontecer, o bibliotecário usa `registrarDano(devolucao, descricao)`. O sistema cria um `RegistroDano` ligado àquela devolução, anotando um valor estimado de conserto. Imediatamente, a situação do exemplar físico muda para **DANIFICADO**. O material não afirma que o valor estimado do dano gera automaticamente uma cobrança na conta de débitos, mostrando que multas por atraso e registros de danos são tratados por classes diferentes.

> Em resumo, o sistema funciona conectando classes que representam informações (como Usuário, Obra e Exemplar), registros de histórico (Empréstimo, Renovação e Devolução) e serviços que aplicam as regras (Autenticação, Empréstimo e Multa). Isso garante o controle completo do acervo e dos usuários. Muito obrigado pela atenção!